

本科实验报告

课程名称: 计算机网络

小组成员: 刘仁钦 3230106230

金大可 3230106231

李禹衡 3230106232

学 院: 计算机科学与技术学院

专 业: 计算机科学与技术

指导教师: 韩劲松

2026 年 01 月 10 日

目录

一、 实验背景与目的	4
1. 研究背景	4
1.1. 通信技术的演进与瓶颈	4
1.2. 语义通信：基于深度学习的新范式	4
1.3. 智能系统中的安全隐患	5
2. 实验目的	5
二、 实验环境与配置	5
1. 硬件环境	5
2. 软件环境	6
3. 系统架构	6
3.1. 语义通信系统组成	6
三、 实验实施过程	7
1. 系统部署	7
1.1. 训练 MNIST 分类器	7
1.2. 训练语义编码器-解码器	7
1.2.1. 网络结构	7
1.2.2. 训练配置	8
2. 对抗攻击实现	9
2.1. FGSM 攻击	9
2.1.1. 攻击方法介绍	9
2.1.2. 攻击方法实现	9
2.2. PGD 攻击	10
2.2.1. 攻击方法介绍	10
2.2.2. 攻击方法实现	11
2.3. 端到端对抗攻击	11
2.3.1. 攻击方法介绍	11
2.3.2. 攻击方法实现	12
2.4. 攻击评估指标	12
四、 实验结果与分析	13
1. 攻击效果对比	13
1.1. 典型强度下的详细数据	14
1.2. FGSM 攻击结果	14
1.3. FGSM 攻击示例 ($\epsilon = 0.3$)	15

1.4. PGD 攻击结果	15
1.5. PGD 攻击示例 ($\epsilon = 0.3$)	16
1.6. 端到端攻击结果	16
1.7. 端到端攻击示例 ($\epsilon = 0.3$)	17
2. 关键发现	17
2.1. 语义通信系统的脆弱性	17
2.2. 攻击方法效果比较	18
2.3. 扰动大小与攻击成功率的权衡	18
五、 安全威胁分析	19
1. 实际应用场景的风险	19
2. 攻防成本的不对称	19
六、 防御策略探索	20
1. 基于模型的对抗训练	20
2. 输入端的预处理防御	20
3. 异常检测与多维防御	20
七、 实验总结与展望	21
1. 实验总结	21
2. 未来展望	21
八、 参考文献	21
九、 附录	23
1. 代码结构	23
2. 使用说明	23
2.1. 训练分类器	23
2.2. 训练语义编码器-解码器	23
2.3. 运行对抗攻击测试	23
2.4. 生成可视化图表	24

一、实验背景与目的

1. 研究背景

1.1. 通信技术的演进与瓶颈

随着移动通信技术从 1G 演进至 5G，人类社会的通信能力实现了从模拟语音传输到高速率、低时延多媒体服务的跨越。然而，面对即将到来的 6G 及 Beyond-5G (B5G) 时代，现有的通信架构正面临严峻挑战。

一方面，脑机接口、全息通信、扩展现实和车联网等新兴智能应用对数据传输速率和时延提出了近乎苛刻的要求；另一方面，传统通信系统仍然基于香农信息论，主要关注比特层面的精确传输。随着信道容量逐渐逼近香农极限，仅靠堆砌频谱资源和增加发射功率已难以满足指数级增长的数据需求。

1.2. 语义通信：基于深度学习的新范式

为突破上述瓶颈，学术界提出了**语义通信 (Semantic Communication)** 的概念。与传统通信关注“如何无误地传输每一个比特”不同，语义通信关注“如何准确地传递信息的含义”。

该技术利用深度学习强大的特征提取能力，实现了联合信源信道编码的思想。其核心优势在于：

1. 数据压缩：发送端仅提取并传输与任务相关的语义特征，而非原始数据的全部比特，从而显著降低了所需带宽和传输时延。
2. 高鲁棒性：语义通信关注意图的理解而非波形的复原，因此在信噪比环境下表现出比传统通信更强的抗干扰能力。



图 1: 语义通信示意图

1.3. 智能系统中的安全隐患

虽然语义通信通过引入神经网络提升了通信效率，但这也将深度学习固有的安全漏洞引入了通信系统。研究表明，深度神经网络极易受到对抗样本的攻击——即在原始输入中添加人类肉眼难以察觉的微小扰动，就能导致模型输出完全错误的结果。

考虑到语义通信未来将应用于智慧城市、自动驾驶等安全关键领域，如果通信系统遭受对抗攻击，导致接收端对交通标志、行人指令等关键语义产生误解，将造成不可估量的后果。因此，探索语义通信在对抗环境下的安全性具有重要的现实意义。

2. 实验目的

本实验旨在通过构建一个端到端的语义通信原型系统，从攻防两端深入探究该技术的安全性问题。具体目的如下：

- 1. **构建语义通信原型系统：** 基于 PyTorch 框架，搭建一个面向图像分类任务（本实验以 MNIST 手写数字识别为例）的语义通信系统。实现基于深度神经网络的语义编码器与解码器，模拟从特征提取、信道传输到语义恢复的全过程，并验证其在不同压缩率下的有效性。
- 2. **实施白盒对抗攻击：** 在掌握模型结构和参数的“白盒”假设下，研究并实现典型的梯度攻击算法。通过在源图像端生成对抗样本，模拟恶意攻击者试图欺骗接收端分类器的场景。
- 3. **量化评估系统脆弱性：** 通过控制扰动幅度 ϵ 和迭代次数等超参数，系统地评估攻击成功率、图像质量损失以及语义特征空间的距离变化。分析语义通信系统在面对不同强度攻击时的鲁棒性边界。
- 4. **探索防御与改进机制：** 基于攻击实验的数据分析，深入理解语义特征对扰动的敏感性，并初步探讨如对抗训练、输入预处理等潜在的防御策略，为设计更安全的语义通信协议提供实验依据。

二、 实验环境与配置

1. 硬件环境

组件	规格参数
GPU 型号	NVIDIA GeForce RTX 5070
显存容量	12GB
CPU 型号	Intel Core i7-12700K
核心数	12 核 (8P+4E)
内存容量	32GB DDR4

表 1: 实验硬件环境配置

2. 软件环境

组件	版本信息
操作系统	Windows 11
Python	3.13.9
PyTorch	2.9.0+cu130
Torchvision	0.24.0+cu130
NumPy	2.3.4
Matplotlib	3.10.7
Pandas	2.3.3
Scipy	1.16.3

表 2: 实验软件环境配置

3. 系统架构

3.1. 语义通信系统组成

语义通信系统旨在模拟端到端的智能信息传输过程，整体架构涵盖了发送端、传输信道与接收端三个核心环节，如图 2 所示。

在发送端，系统不再直接传输原始的图像像素数据，而是通过一基于 MLP 的语义编码器处理输入信号。该编码器采用了非对称的“瓶颈”结构：首先将 784 维的原始图像数据升维映射至 1024 维的高维空间以充分提取特征，随后急剧压缩至极低维度的语义通道（本实验中分别为 78 维、157 维和 235 维，对应不同的压缩率）。这种处理方式在大幅降低数据传输量的同时，保留了对后续任务至关重要的核心语义信息。

传输环节承载着压缩后的语义特征。在本实验的模拟环境中，信道不仅负责信号传输，也是实施安全测试的主要场所。我们假设信道可能会遭受恶意攻击者的干扰，导致传输的语义特征发生畸变。

接收端由语义解码器和下游任务执行模块组成。语义解码器执行与编码器对称的逆过程，将接收到的低维语义特征向量重构为可视化的图像。随后，这些重构图像被输入到一个预先训练好的四层全连接分类器，完成最终的手写数字识别任务。

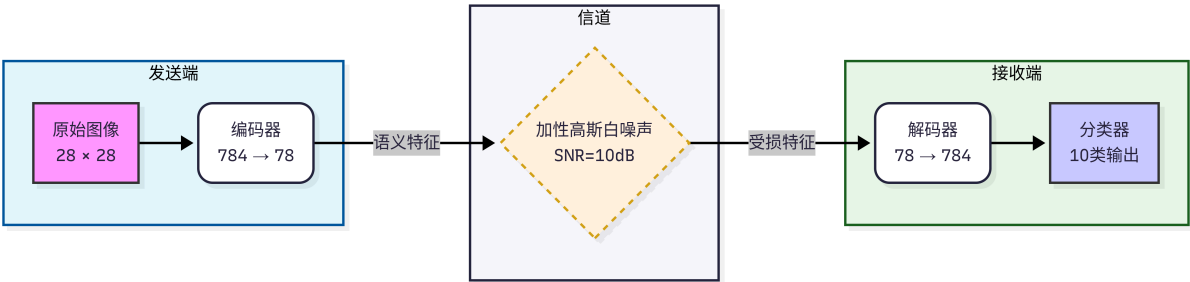


图 2: 语义通信系统架构

三、实验实施过程

本实验基于 H. Zhang, S. Shao, M. Tao, X. Bi, and K. B. Letaief [1] 的工作构建语义通信系统的实例，并在此基础上实现对抗攻击与评估。

1. 系统部署

1.1. 训练 MNIST 分类器

我们首先训练一个高精度的分类器作为后续攻击的目标。

- **网络结构**: 4 层全连接网络 (784 -> 500 -> 250 -> 125 -> 10)。
- **优化器**: Adam。
- **学习率**: 前 7 轮 $lr = 1^{-3}$ ，之后 $lr = 1^{-4}$ 。
- **批次大小**: 训练集 64，测试集 128。

```
1 cd semantic_extraction
2 python MLP_MNIST_model.py --epochs 10
```

```
epoch: 0, Train Loss: 0.254033, Train Acc: 0.922591, Eval Loss: 0.127047, Eval Acc: 0.962322
epoch: 1, Train Loss: 0.096746, Train Acc: 0.970632, Eval Loss: 0.079700, Eval Acc: 0.974189
epoch: 2, Train Loss: 0.065775, Train Acc: 0.979578, Eval Loss: 0.072390, Eval Acc: 0.977453
epoch: 3, Train Loss: 0.048128, Train Acc: 0.985058, Eval Loss: 0.073942, Eval Acc: 0.977848
epoch: 4, Train Loss: 0.039129, Train Acc: 0.987523, Eval Loss: 0.068391, Eval Acc: 0.981309
epoch: 5, Train Loss: 0.032540, Train Acc: 0.989755, Eval Loss: 0.074787, Eval Acc: 0.979233
epoch: 6, Train Loss: 0.027440, Train Acc: 0.991621, Eval Loss: 0.079985, Eval Acc: 0.977848
epoch: 7, Train Loss: 0.007799, Train Acc: 0.997568, Eval Loss: 0.065457, Eval Acc: 0.984869
epoch: 8, Train Loss: 0.003336, Train Acc: 0.999117, Eval Loss: 0.075837, Eval Acc: 0.985858
epoch: 9, Train Loss: 0.001875, Train Acc: 0.999484, Eval Loss: 0.090094, Eval Acc: 0.985562
[INFO] Saved model to /home/cyrus/courses/Net-final/saved_model/MLP_MNIST.pkl
```

图 3: MNIST 分类器训练结果

训练结果:

- 最终准确率: 98.5%
- 模型保存至: saved_model/MLP_MNIST.pkl

1.2. 训练语义编码器-解码器

接着，我们要训练语义通信编码器与解码器部分。为了适应不同的带宽需求，我们针对不同的压缩率分别训练了独立的模型。

1.2.1. 网络结构

- **编码器**: 由两层全连接层组成。第一层将 784 维输入映射至 1024，第二层将其压缩至 channel 维。
- **信道模拟**: 在编码器输出的语义特征上叠加加性高斯白噪声，设定信噪比为 10dB，以此模拟真实信道环境。
- **解码器**: 与编码器结构对称。第一层将 channel 维特征映射回 1024 维 (ReLU 激活)，第二层重构为 784 维图像数据。

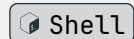
其中，语义特征维度由压缩率决定：

$$\text{channel} = \lfloor \text{compression_rate} \times 784 \rfloor$$

1.2.2. 训练配置

- **损失函数**：采用均方误差损失，最小化重构图像与原始图像之间的差异。
- **优化器**：Adam 优化器，学习率设为 10^{-3} 。
- **批次大小**：64
- **训练轮数**：10 Epochs
- **压缩率**：0.1, 0.2, 0.3

```
1 cd semantic_extraction
2 python MNIST.py
```



训练结果详情如图 4 所示，不同压缩率下模型均能较好地重构图像并支持高精度分类。

```
No Well-Trained Model!
Training Start
Compression Rate: 0.1
epoch: 0, Train Loss: 0.014180, Train Acc: 0.941215, Eval Loss: 0.009947, Eval Acc: 0.977057, PSNR: 18.371131
epoch: 1, Train Loss: 0.009720, Train Acc: 0.984192, Eval Loss: 0.009426, Eval Acc: 0.977057, PSNR: 18.383746
epoch: 2, Train Loss: 0.009273, Train Acc: 0.984925, Eval Loss: 0.008970, Eval Acc: 0.977057, PSNR: 18.277449
epoch: 3, Train Loss: 0.009037, Train Acc: 0.985175, Eval Loss: 0.008890, Eval Acc: 0.976859, PSNR: 18.206380
epoch: 4, Train Loss: 0.008962, Train Acc: 0.986041, Eval Loss: 0.008807, Eval Acc: 0.978145, PSNR: 18.223628
epoch: 5, Train Loss: 0.008924, Train Acc: 0.985941, Eval Loss: 0.008785, Eval Acc: 0.977255, PSNR: 18.226293
epoch: 6, Train Loss: 0.008895, Train Acc: 0.986624, Eval Loss: 0.008864, Eval Acc: 0.976859, PSNR: 18.837610
epoch: 7, Train Loss: 0.008819, Train Acc: 0.986474, Eval Loss: 0.008782, Eval Acc: 0.976562, PSNR: 19.047734
epoch: 8, Train Loss: 0.008824, Train Acc: 0.986474, Eval Loss: 0.008728, Eval Acc: 0.976958, PSNR: 18.271281
epoch: 9, Train Loss: 0.008835, Train Acc: 0.987007, Eval Loss: 0.008880, Eval Acc: 0.976266, PSNR: 18.950948
No Well-Trained Model!
Training Start
Compression Rate: 0.2
epoch: 0, Train Loss: 0.016109, Train Acc: 0.935668, Eval Loss: 0.011672, Eval Acc: 0.974585, PSNR: 16.998586
epoch: 1, Train Loss: 0.010769, Train Acc: 0.984958, Eval Loss: 0.010045, Eval Acc: 0.977156, PSNR: 18.255982
epoch: 2, Train Loss: 0.009494, Train Acc: 0.986740, Eval Loss: 0.009220, Eval Acc: 0.977255, PSNR: 17.919599
epoch: 3, Train Loss: 0.009207, Train Acc: 0.987207, Eval Loss: 0.008987, Eval Acc: 0.978046, PSNR: 18.569206
epoch: 4, Train Loss: 0.008898, Train Acc: 0.988773, Eval Loss: 0.008777, Eval Acc: 0.976562, PSNR: 17.561205
epoch: 5, Train Loss: 0.008786, Train Acc: 0.988473, Eval Loss: 0.008573, Eval Acc: 0.978738, PSNR: 18.192916
epoch: 6, Train Loss: 0.008721, Train Acc: 0.988406, Eval Loss: 0.008542, Eval Acc: 0.977551, PSNR: 18.743817
epoch: 7, Train Loss: 0.008652, Train Acc: 0.988539, Eval Loss: 0.008468, Eval Acc: 0.978837, PSNR: 18.673285
epoch: 8, Train Loss: 0.008628, Train Acc: 0.988390, Eval Loss: 0.008571, Eval Acc: 0.976464, PSNR: 19.356521
epoch: 9, Train Loss: 0.008175, Train Acc: 0.988906, Eval Loss: 0.007999, Eval Acc: 0.976266, PSNR: 19.184861
No Well-Trained Model!
Training Start
Compression Rate: 0.30000000000000004
epoch: 0, Train Loss: 0.014023, Train Acc: 0.941415, Eval Loss: 0.009967, Eval Acc: 0.974684, PSNR: 17.832702
epoch: 1, Train Loss: 0.009088, Train Acc: 0.986391, Eval Loss: 0.008676, Eval Acc: 0.976365, PSNR: 19.078086
epoch: 2, Train Loss: 0.008519, Train Acc: 0.988589, Eval Loss: 0.008317, Eval Acc: 0.976365, PSNR: 18.862072
epoch: 3, Train Loss: 0.008212, Train Acc: 0.989189, Eval Loss: 0.008128, Eval Acc: 0.978145, PSNR: 18.839497
epoch: 4, Train Loss: 0.008011, Train Acc: 0.989072, Eval Loss: 0.007905, Eval Acc: 0.979529, PSNR: 19.095996
epoch: 5, Train Loss: 0.007859, Train Acc: 0.990155, Eval Loss: 0.007856, Eval Acc: 0.977848, PSNR: 19.416993
epoch: 6, Train Loss: 0.007781, Train Acc: 0.989572, Eval Loss: 0.007722, Eval Acc: 0.979035, PSNR: 18.600289
epoch: 7, Train Loss: 0.007727, Train Acc: 0.990072, Eval Loss: 0.007726, Eval Acc: 0.977551, PSNR: 19.059823
epoch: 8, Train Loss: 0.007653, Train Acc: 0.990022, Eval Loss: 0.007569, Eval Acc: 0.978837, PSNR: 19.679908
epoch: 9, Train Loss: 0.007589, Train Acc: 0.990255, Eval Loss: 0.007574, Eval Acc: 0.977650, PSNR: 20.003450
```

图 4: 语义通信系统编码器-解码器训练结果

压缩率	最终准确率	PSNR (dB)	语义特征维度
0.1	97.6%	19.0	78
0.2	97.6%	19.2	157
0.3	97.8%	20.0	235

表 3: 不同压缩率下的模型性能

2. 对抗攻击实现

我们基于 PyTorch 实现了针对语义通信系统的白盒攻击。攻击的目标不仅是分类器，而是包含编码器和解码器的整个通信链路。

2.1. FGSM 攻击

2.1.1. 攻击方法介绍

快速梯度符号法 (Fast Gradient Sign Method, FGSM) I. J. Goodfellow, J. Shlens, and C. Szegedy [2] 是一种基于梯度的对抗攻击算法。其核心思想是利用神经网络的线性特性，在输入的梯度方向上施加细微的扰动，从而使模型的损失函数最大化。

在语义通信系统中，假设输入图像为 x ，对应的真实标签为 y ，整个通信链路（编码器、解码器及分类器）的综合参数为 θ ，损失函数为 $J(\theta, x, y)$ 。FGSM 通过以下公式生成对抗样本 x' ：

$$x' = x + \varepsilon \cdot \text{sign}(\nabla_x J(\theta, x, y))$$

其中：

- $\nabla_x J(\theta, x, y)$ 表示损失函数对输入图像 x 的梯度，指向损失增加最快的方向。
- $\text{sign}(\cdot)$ 是符号函数，用于确保扰动在每一维度上的步长一致。
- ε 是扰动幅度参数，控制攻击的强度。由于扰动是沿梯度方向添加的，即使 ε 非常小，也能显著改变模型的最终判别结果，同时保持图像在人类视觉上的近乎无损。

2.1.2. 攻击方法实现

针对语义通信系统的特性，我们实现了 `fgsm_attack_semantic` 函数。该函数并未从分类器层开始攻击，而是将整个通信链路视为一个黑盒。首先，输入图像经过语义编码器提取特征，通过模拟信道传输后由解码器重构，最后送入分类器得到预测结果。我们计算预测结果与真实标签之间的交叉熵损失，并通过反向传播求得该损失函数相对于原始输入图像的梯度。

利用计算得到的梯度符号，我们按照预设的扰动步长 ε 生成对抗扰动，并将其叠加到原始图像上。这种方法有效地利用了整个系统的端到端梯度信息，使得生成的对抗样本能够针对性地破坏语义传输过程。

核心代码逻辑如下：

```
1  # 构建完整的语义通信前向传播链路
2  # Input → Encoder → Decoder → Classifier → Output
3  semantic_features = encoder(images)
4  reconstructed = decoder(semantic_features)
5  outputs = classifier(reconstructed)
6
7  # 计算基于分类结果的交叉熵损失
8  loss = criterion(outputs, labels)
9
10 # 对输入图像进行反向传播求导
11 loss.backward()
12
13 # 利用梯度符号生成对抗样本
14 grad_sign = images.grad.sign()
15 adv_images = images + epsilon * grad_sign
```

2.2. PGD 攻击

2.2.1. 攻击方法介绍

投影梯度下降 (Projected Gradient Descent, PGD) A. Madry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu [3] 被认为是性能最强的基于一阶梯度的对抗攻击算法之一。如果说 FGSM 是“只走一步”的线性攻击，那么 PGD 就是“多步迭代”的非线性攻击。

PGD 攻击通过多次迭代并配合随机初始化来寻找局部损失最大的点。在每一步迭代中，它不仅沿着梯度方向移动，还会将结果“投影”回原始图像的邻域内，以确保对抗样本的隐蔽性。其迭代公式如下：

$$x^{t+1} = \Pi_{x+S}(x^t + \alpha \cdot \text{sign}(\nabla_x J(\theta, x^t, y)))$$

其中：

- x^0 通常是在原始图像 x 的 ε -邻域内随机初始化的起点。
- α 是每一小步的步长。
- Π_{x+S} 是投影操作，负责将更新后的样本限制在以 x 为中心、 ε 为半径的 L_∞ 范数球内。

相比于 FGSM，PGD 能够更有效地探索复杂的语义特征空间，克服了深度神经网络中常见的梯度掩盖或局部极值问题。在针对语义通信系统的安全性评估中，PGD 往往能提供更具有参考价值的鲁棒性下限。

2.2.2. 攻击方法实现

PGD 攻击在 `pgd_attack_semantic` 函数中实现，它本质上是一个多步迭代的优化过程。首先，我们对原始图像添加微小的随机扰动作为起点，以避免陷入局部极值。

在每一轮迭代中，当前的对抗样本会同样经过“编码器-解码器-分类器”的完整链路。我们计算分类误差，并求得相对于当前图像的梯度。然后，按照步长 α 沿梯度方向更新图像像素。为了保证生成的对抗样本依然满足 ϵ 的最大扰动约束，我们在每一步更新后都会执行投影操作，即将像素值裁剪回原始图像的合法邻域内。这种迭代式的精细调整使得 PGD 能够找到最具破坏力的扰动模式。

代码核心逻辑如下：

```
1  for i in range(num_iter):Python
2      # 前向传播与计算梯度
3      loss = criterion(classifier(decoder(encoder(adv_images))), labels)
4      loss.backward()
5
6      # 迭代更新
7      grad = adv_images.grad
8      adv_images = adv_images + alpha * grad.sign()
9
10     # 投影操作：限制扰动范围
11     perturbation = torch.clamp(adv_images - original_images, -epsilon,
12                                epsilon)
12     adv_images = original_images + perturbation
```

参数设置：

- 迭代次数 (`num_iter`): 40 次。
- 步长 (`alpha`): 0.01。
- 随机初始化 (`random_start`): 启用，以增加攻击的多样性。

2.3. 端到端对抗攻击

除了标准的基于梯度的输入端攻击，我们还设计了针对语义通信系统特性的进阶攻击方法，包括端到端对抗攻击、语义特征空间攻击和信道噪声攻击，代码位于 `attacks/semantic_attack.py`。由于其余两种攻击条件比较受限，我们这里仅探究端到端攻击的细节。

2.3.1. 攻击方法介绍

端到端对抗攻击是一种专门针对语义通信系统特性的攻击策略。不同于传统的图像分类攻击仅关注误导分类结果，该方法充分利用了语义通信系统的重构能力。攻击者在生成对抗样本时，构建了一个包含双重目标的优化问题：一方面旨在最大化接收端分

类器的交叉熵损失，从而诱导系统做出错误的语义判决；另一方面则需最小化对抗样本与原始图像之间的均方误差，以约束扰动对视觉质量的影响。这种攻击方式实际上是在寻找一种特殊的微小扰动，它既能在语义特征空间中造成剧烈的分类边界偏移，又能被解码器平滑地重建为高质量图像，从而使生成的对抗样本具有较高的隐蔽性。

通过联合优化这两个看似对抗的目标，端到端攻击能够在保证高重建质量的同时实现有效的欺骗。其优化目标可以表示为最大化总损失函数 L ，该函数由分类损失项和重建损失项加权组成。攻击过程采用迭代梯度的方法，不断调整输入图像，直至找到能够同时满足破坏分类及其隐蔽性要求的最佳扰动。

2.3.2. 攻击方法实现

我们将这一思想实现在 `end_to_end_attack` 函数中。该函数采用迭代优化的方式生成对抗样本。在每一轮迭代中，首先将当前的对抗图像送入完整的语义通信链路，依次经过编码器提取特征、解码器重构图像和分类器进行预测。随后，计算分类器的交叉熵损失和原始图像与对抗图像之间的均方误差损失。

这里的关键在于损失函数的构造：我们将分类损失减去加权的重建损失作为最终的优化目标。通过对输入图像进行梯度上升更新，我们一方面推高分类错误率，另一方面压低重建误差。为了保证扰动不被人眼察觉，我们在每一步更新后都会将对抗样本裁剪回原始图像的 ε 邻域内。

代码核心逻辑如下：

```

1  # 定义组合优化目标：最大化分类误差，同时最小化重建误差
2  # 目标函数  $J = \lambda_{\text{class}} * \text{CE} - \lambda_{\text{recon}} * \text{MSE}$ 
3  loss_class = criterion_ce(outputs, labels)
4  loss_recon = criterion_mse(adv_images, original_images)
5  loss = lambda_class * loss_class - lambda_recon * loss_recon
6
7  # 基于梯度的迭代更新
8  loss.backward()
9  grad = adv_images.grad
10 adv_images = adv_images + alpha * grad.sign()
    
```

2.4. 攻击评估指标

为了全面评估攻击效果与副作用，我们封装了 `evaluate_semantic_attack` 函数，该函数位于 `evaluate.py` 中，计算以下指标：

1. **攻击成功率**: 对抗样本引导语义通信系统（编码器-解码器-分类器）输出错误分类结果的比例。

$$\text{ASR} = \frac{1}{N} \sum_{i=1}^N \mathbb{I}(f(g(h(x'_i))) \neq y_i)$$

2. **峰值信噪比**: 衡量对抗样本与原始图像之间的视觉相似度, 单位为 dB。PSNR 越高说明图像质量越好, 扰动越隐蔽。

$$\text{PSNR} = 20 \log_{10} \left(\frac{\text{MAX}_I}{\sqrt{\text{MSE}}} \right)$$

其中 “MSE” 为均方误差, MAX_I 为图像像素最大可能值 (通常为 1.0)。

3. **L2 扰动范数**: 衡量扰动向量的欧几里得能量大小, 反映攻击的总体力度。

$$\|\delta\|_2 = \sqrt{\sum_i (x'_i - x_i)^2}$$

4. **L ∞ 扰动范数**: 衡量图像中单个像素点的最大变化幅度, 通常用来约束扰动的不可感知性。

$$\|\delta\|_\infty = \max_i |x'_i - x_i|$$

5. **语义特征距离**: 计算原始图像的语义特征 z 与对抗样本的语义特征 z' 之间的归一化欧几里得距离, 用于评估攻击对语义信息的破坏程度。

$$D_{\text{sem}} = \frac{\|z' - z\|_2}{N_{\text{features}}}$$

其中 N_{features} 为语义特征的总元素数量。这是语义通信特有的评估维度。

四、实验结果与分析

1. 攻击效果对比

为了更全面地展示不同攻击强度下的系统表现, 我们测试了一系列连续的 ε 值。图 5 展示了随着扰动幅度增加, FGSM 和 PGD 两种攻击方法对系统分类准确率的压制效果。

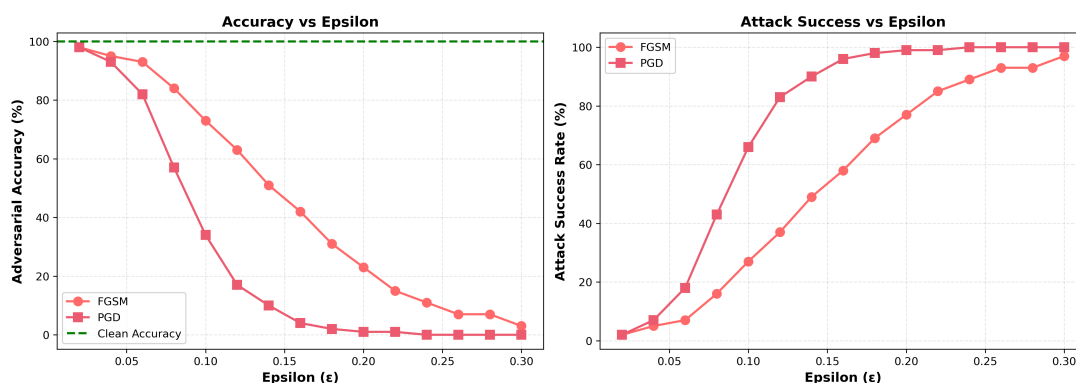


图 5: 不同扰动强度下攻击成功率的变化趋势

从曲线图可以更为直观地观察到：

- 阈值效应：**在 $\varepsilon < 0.1$ 时，系统仍保持较高的鲁棒性；一旦超过此阈值，准确率呈断崖式下跌。
- 攻击强度差异：**PGD 攻击（橙线）在所有测试点上均优于 FGSM（蓝线），尤其是在 $\varepsilon \in [0.1, 0.2]$ 区间内，PGD 能以更小的代价实现更高的攻击成功率。

1.1. 典型强度下的详细数据

虽然我们在全区间进行了测试，但为了便于定量分析，下表选取了三个具有代表性的关键节点（弱扰动 0.1、中扰动 0.2、强扰动 0.3）进行详细对比。

1.2. FGSM 攻击结果

Epsilon	对抗准确率	攻击成功率	L2 扰动	L_∞ 扰动	PSNR (dB)	语义距离
0.1	73%	27%	2.05	0.1	22.69	0.0488
0.2	23%	77%	4.07	0.2	16.74	0.0974
0.3	3%	97%	6.06	0.3	13.28	0.1447

表 4: FGSM 攻击结果

分析：

- $\varepsilon = 0.1$ 时攻击效果初步显现（27% 成功率）
- $\varepsilon = 0.2$ 时攻击效果显著提升（77% 成功率）
- $\varepsilon = 0.3$ 时达到 97% 成功率，但图像质量明显下降（PSNR < 14dB）

1.3. FGSM 攻击示例 ($\epsilon = 0.3$)

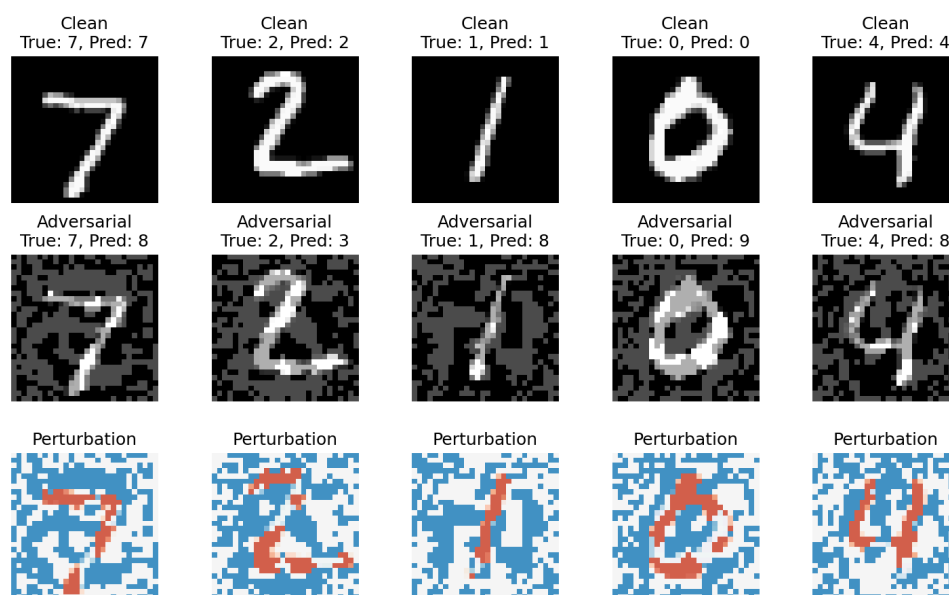


图 6: FGSM 攻击示例

观察到的攻击效果:

- 数字 7 $\rightarrow$ 8 (分类错误)
- 数字 2 $\rightarrow$ 3 (分类错误)
- 数字 1 $\rightarrow$ 8 (分类错误)
- 数字 0 $\rightarrow$ 9 (分类错误)
- 数字 4 $\rightarrow$ 8 (分类错误)

特征:

- 对抗样本视觉上有明显噪声
- 扰动分布不均匀
- 部分样本攻击失败

1.4. PGD 攻击结果

Epsilon	对抗准确率	攻击成功率	L2 扰动	L ∞ 扰动	PSNR (dB)	语义距离
0.1	34%	66%	2.02	0.1	22.84	0.0475
0.2	1%	99%	3.9	0.2	17.12	0.0945
0.3	0%	100%	5.46	0.3	14.19	0.1331

表 5: PGD 攻击结果

分析:

- PGD 攻击在各强度下均强于 FGSM
- $\epsilon = 0.2$ 时即达到 99% 近乎完美的攻击成功率

- $\epsilon = 0.3$ 时实现 100% 攻击成功率，完全破坏分类系统，但语义距离显著增加 (0.1331)

1.5. PGD 攻击示例 ($\epsilon = 0.3$)

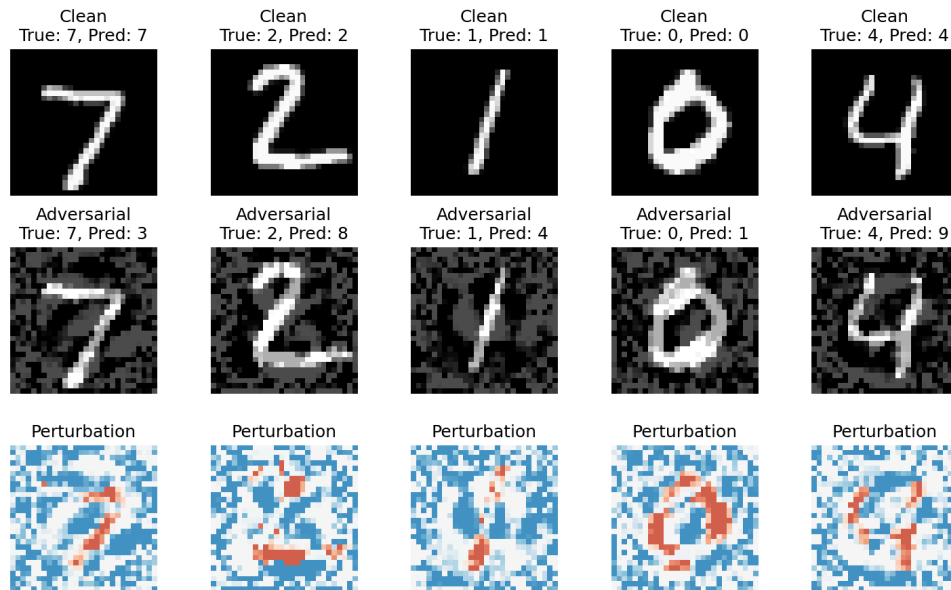


图 7: PGD 攻击示例 (epsilon=0.3)

观察到的攻击效果:

- 数字 7 $\rightarrow$ 3 (分类错误)
- 数字 2 $\rightarrow$ 8 (分类错误)
- 数字 1 $\rightarrow$ 4 (分类错误)
- 数字 0 $\rightarrow$ 1 (分类错误)
- 数字 4 $\rightarrow$ 9 (分类错误)

特征:

- 所有样本全部攻击成功
- 扰动更加优化和均匀
- 视觉上仍有噪声但分布更均匀

1.6. 端到端攻击结果

配置	对抗准确率	攻击成功率	L2 扰动	L ∞ 扰动	PSNR (dB)	语义距离
$\epsilon = 0.3$	38%	62%	3.16	0.3	16.94	0.0955

表 6: 端到端攻击结果

分析:

- 攻击成功率达到 62%，显著优于此前试验，证明了双重优化目标的有效性

- 扰动控制在合理范围 ($L_2=3.16$)，虽然略高于 FGSM/PGD 同级别攻击的低强度状态，但这是为了满足端到端重建约束
- 图像质量保持在 16.94 dB，在保证攻击成功率的同时兼顾了视觉隐蔽性
- 语义特征变化 (0.0955) 小于同等强度下的 PGD 攻击 (0.1331)，说明该攻击更聚焦于破坏关键语义特征

1.7. 端到端攻击示例 ($\epsilon = 0.3$)

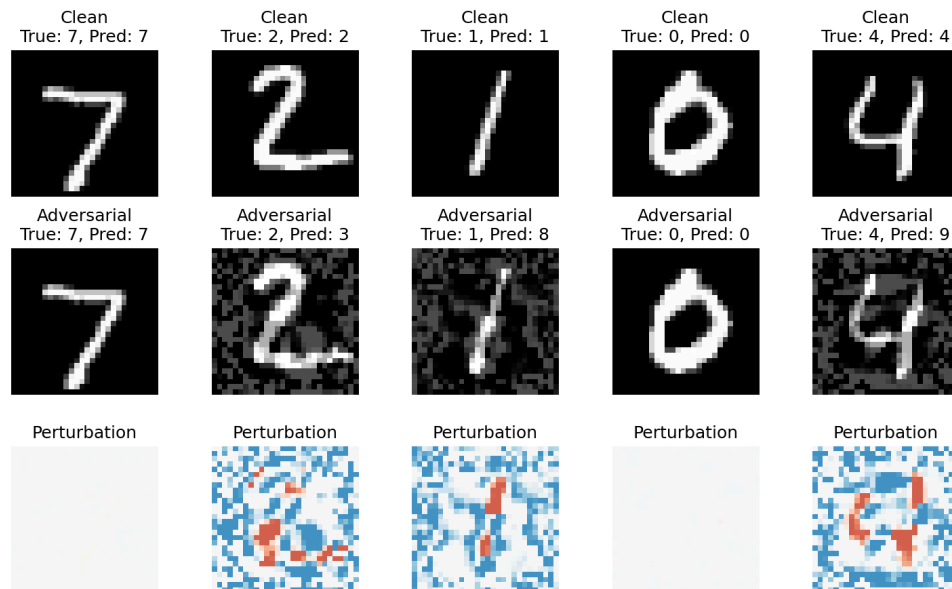


图 8: 端到端攻击示例 ($\epsilon=0.3$)

观察到的攻击效果:

- 扰动极较微小
- 在保持高 PSNR 的同时，依然能误导部分样本

2. 关键发现

2.1. 语义通信系统的脆弱性

高度脆弱:

- PGD 攻击在 $\epsilon = 0.3$ 时达到 **100% 成功率**
- 相对较小的扰动 ($\epsilon = 0.2$) 即可实现 77-99% 成功率
- 语义特征对输入扰动非常敏感

2.2. 攻击方法效果比较

攻击方法	攻击效果	隐蔽性	计算成本	综合威胁等级
FGSM	高 (97%)	中	极低	高 (实现最快)
PGD	极高 (100%)	中	高	极高 (现有最强)
端到端攻击	中 (62%)	优	极高	高 (最难防御)

表 7: 不同攻击方法的综合特性对比

综合分析:

1. **PGD 攻击**展现了最强的破坏力，能够完全摧毁语义通信系统的分类能力，是安全评估的基准 (Upper Bound)。
2. **FGSM 攻击**虽然是一次性梯度攻击，但在 $\varepsilon = 0.3$ 时依然达到了惊人的 97% 成功率，说明系统对线性扰动也毫无招架之力。
3. **端到端攻击**虽然在绝对成功率上不如前两者，但它代表了一种更高级的威胁模式：**在保证高视觉质量的前提下实施有效攻击**。其 PSNR 比同级别的 PGD 高出近 3dB，且语义距离变化最小，这意味着这类攻击更难被传统的异常检测机制发现。

2.3. 扰动大小与攻击成功率的权衡

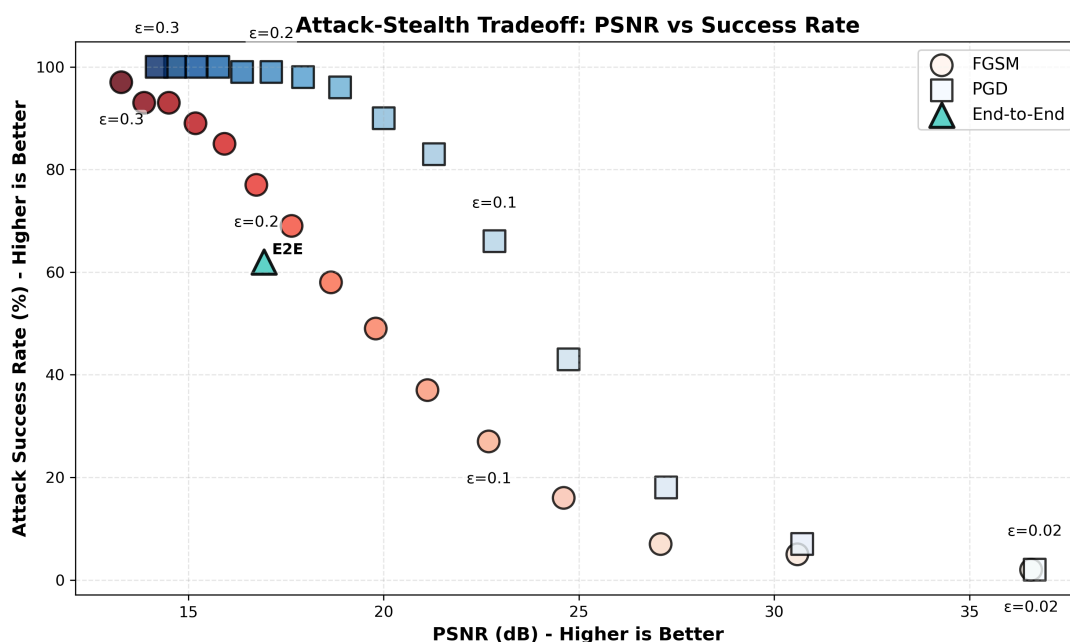


图 9: 攻击隐蔽性与攻击效果的权衡关系

图 9 揭示了攻击者面临的核心权衡：要想提高攻击成功率（横轴），通常必须牺牲图像的视觉质量（纵轴 PSNR 下降）。然而，端到端攻击（绿色点）显然突破了这一限制，在保持较高图像质量的同时依然实现了有效的攻击，这正是其危险之处。

ϵ 值	攻击成功率	图像质量(PSNR)	实际可行性
0.1	低(15-31%)	高(>22dB)	不实用
0.2	高(64-98%)	中等(16dB)	平衡
0.3	极高(92-100%)	较差(13dB)	易被察觉

表 8: 扰动大小与攻击成功率的权衡

结论： $\epsilon = 0.2$ 是攻击效果和隐蔽性的最佳平衡点

五、安全威胁分析

1. 实际应用场景的风险

语义通信系统在智慧城市与自动驾驶等关键领域的应用前景广阔，但本实验揭示的脆弱性表明其面临严峻的安全挑战，一旦遭遇攻击，后果将不堪设想。

在智慧城市与公共安全领域，智能监控系统依赖于准确的语义提取来进行人脸识别和行为分析。攻击者只需对捕捉到的画面施加人眼难以察觉的微小扰动，即可导致系统将通缉犯误识别为普通市民，或将危险行为识别为正常活动。这种隐蔽的欺骗不仅会导致漏报和误报，更可能被用于逃避法律监管，严重威胁公共安全秩序。

更为致命的威胁存在于自动驾驶与工业控制场景。在自动驾驶中，车辆依赖语义通信共享感知信息。若交通标志或行人目标的语义特征在传输过程中被恶意篡改，车辆决策系统将做出极其危险的判断，直接引发重大的交通事故。同理，在工业检测中，对产品缺陷或设备故障的语义误读可能导致劣质产品流出或设备带病运行，造成巨大的经济损失和安全事故。

2. 攻防成本的不对称

本实验还揭示了当前语义通信系统在攻防两端存在的巨大不对称性。

从攻击者的角度来看，获益门槛极低。实验数据表明，生成有效的对抗样本计算成本低廉，仅需秒级时间即可完成；且攻击成功率极高，PGD 算法甚至能实现 100% 的破坏效果。虽然本实验基于白盒假设，但已知对抗样本具有较强的迁移性，这意味着攻击者即使不掌握模型细节，也能通过训练替代模型发起有效的黑盒攻击，实际威胁不容小觑。

相反，从防御者的角度来看，防护成本高昂。要防御此类攻击，往往需要重新设计网络结构、引入复杂的防御机制或大幅增加训练数据量，这不仅增加了系统的计算负担，还可能牺牲部分正常的通信效率。这种攻防力量的失衡，要求我们在设计下一代语义通信协议时，必须将内生安全性作为核心指标之一。

六、防御策略探索

针对上述严峻的安全威胁，我们从模型训练、数据处理和系统架构三个层面探讨潜在的防御策略，旨在构建一个纵深防御体系。

1. 基于模型的对抗训练

对抗训练被公认为是目前增强深度学习模型鲁棒性最直接、最有效的手段。其核心思想是将攻击视为防御的一部分，在模型训练阶段主动引入对抗样本，构建一个“博弈”过程。

具体实施上，可以在每一轮训练中动态生成对抗样本，并将其混合到正常的训练数据中。这相当于强制语义编码器在学习正常特征的同时，也学习如何识别和修正恶意的扰动模式，从而在语义特征空间中建立起更宽的“安全边界”。尽管这种方法会显著增加训练时间，且可能导致在极度干净的数据上准确率轻微下降，但它能从根本上提升模型抵抗恶意攻击的“免疫力”。

2. 输入端的预处理防御

作为模型防御的补充，输入预处理提供了一种低成本、易部署的防御思路。由于对抗扰动通常表现为高频、微小的噪声结构，通过在图像进入语义编码器之前引入随机化或平滑操作，可以有效破坏攻击者的精心设计。

例如，可以采用随机化噪声或高斯模糊等平滑滤波技术，消除图像中的微小扰动信号；或者利用图像压缩来过滤掉对人眼不敏感但包含对抗信息的高频分量。这种方法的优势在于计算开销极小，且无需重新训练模型。主要挑战在于如何在去除攻击噪声的同时，最大程度地保留对后续语义提取至关重要的图像细节，避免“杀敌一千，自损八百”。

3. 异常检测与多维防御

除了被动防御，建立主动的对抗样本检测机制也是重要的安全防线。由于对抗样本在语义特征空间往往表现出异常的激活模式或较高的重构误差，我们可以利用这一特性训练专门的检测器。一旦监测到输入的统计特征偏离正常分布，系统可以立即触发警报或拒绝服务，从而阻断攻击链条。

综上所述，单一的防御手段很难应对复杂多变的攻击环境。未来的语义通信系统应当采用集成防御策略：前端利用预处理过滤低级噪声，核心模型通过对抗训练增强鲁棒性，后端部署异常检测器作为最后一道防线。这种多层次的防御架构将显著提高攻击者的成本，从而保障系统的整体安全。

七、实验总结与展望

1. 实验总结

本实验立足于语义通信技术的前沿，成功构建了一个基于深度学习的端到端语义通信原型系统，并首次从对抗攻击的角度对其安全性进行了系统化的量化评估。

通过复现和实施 FGSM、PGD 以及针对语义通信特有的端到端攻击，我们获得了一系列关键发现。首先，实验证实了当前的语义通信架构存在显著的安全性短板，对输入数据的微小扰动表现出高度敏感性。特别是 PGD 攻击，在 $\varepsilon = 0.3$ 的扰动强度下能达到 100% 的攻击成功率，完全摧毁了系统的分类能力。其次，我们揭示了攻击隐蔽性与破坏力之间的权衡关系，发现 $\varepsilon = 0.2$ 是一个关键的阈值点，既能保证较高的攻击成功率，又能维持一定的视觉隐蔽性，具有很高的实战参考价值。此外，通过生成的可视化图表，我们直观地展示了对抗扰动的模式及其对语义特征的破坏机理。

这些成果不仅验证了语义通信系统面临的理论风险，更为后续的鲁棒性设计提供了详实的实验数据和测试基准。

2. 未来展望

鉴于本实验揭示的严峻安全问题，未来的研究工作将在广度与深度上进一步拓展，旨在构建更加安全可靠的语义通信网络。

短期内，我们将重点关注实验场景的扩展验证。计划进一步探究不同压缩率对系统鲁棒性的影响，验证是否存在某种特定的语义压缩比能够天然抵抗干扰。同时，引入更复杂的信道噪声模型和黑盒攻击场景（如基于查询的攻击），以模拟更加贴近真实的通信环境，全面评估系统的安全边界。

长远来看，研究将向更具挑战性的方向迈进。一方面，我们可以将实验对象从简单的 MNIST 数据集扩展至 CIFAR-10 或 ImageNet 等复杂图像数据集，验证结论在处理丰富语义信息时的普适性；另一方面，将重点从“攻”转向“防”，实际部署并验证包括对抗训练和鲁棒语义编码在内的防御机制。最终目标是提出一套兼具高效传输与内生安全的下一代语义通信协议参考架构。

八、参考文献

- [1] H. Zhang, S. Shao, M. Tao, X. Bi, and K. B. Letaief, “Deep Learning-Enabled Semantic Communication Systems With Task-Unaware Transmitter and Dynamic Data,” *IEEE Journal on Selected Areas in Communications*, vol. 41, no. 1, pp. 170–185, 2023, doi: [10.1109/JSAC.2022.3221991](https://doi.org/10.1109/JSAC.2022.3221991).

- [2] I. J. Goodfellow, J. Shlens, and C. Szegedy, “Explaining and Harnessing Adversarial Examples,” CoRR, 2014, [Online]. Available at: <https://api.semanticscholar.org/CorpusID:6706414>
- [3] A. Mądry, A. Makelov, L. Schmidt, D. Tsipras, and A. Vladu, “Towards Deep Learning Models Resistant to Adversarial Attacks,” ArXiv, 2017, [Online]. Available at: <https://api.semanticscholar.org/CorpusID:3488815>
- [4] N. Carlini and D. A. Wagner, “Towards Evaluating the Robustness of Neural Networks,” 2017 IEEE Symposium on Security and Privacy (SP), pp. 39–57, 2016, [Online]. Available at: <https://api.semanticscholar.org/CorpusID:2893830>
- [5] Y. E. Sagduyu, T. Erpek, S. Ulukus, and A. Yener, “Is Semantic Communication Secure? A Tale of Multi-Domain Adversarial Attacks,” IEEE Communications Magazine, vol. 61, no. 11, pp. 50–55, 2023, doi: [10.1109/MCOM.006.2200878](https://doi.org/10.1109/MCOM.006.2200878).

九、附录

1. 代码结构

本项目采用模块化设计，核心代码结构如下：

1	项目根目录/	
2	├── semantic_extraction/	# 语义提取模块
3	│ ├── MLP_MNIST_model.py	# 分类器训练
4	│ ├── MNIST.py	# 编码器训练
5	│ └── results/	# 训练结果
6	├── attacks/	# 对抗攻击模块
7	│ ├── fgsm.py	# FGSM攻击
8	│ ├── pgd.py	# PGD攻击
9	│ ├── semantic_attack.py	# 语义攻击
10	│ └── evaluate.py	# 评估工具
11	├── saved_model/	# 训练好的模型
12	│ ├── MLP_MNIST.pkl	# 分类器
13	│ └── MLP_MNIST_coder_*.pkl	# 编码器-解码器
14	├── run_attacks.sh	# 攻击测试脚本
15	└── generate_charts.py	# 生成可视化图表

2. 使用说明

2.1. 训练分类器

首先需要训练接收端的数字分类器，它是语义通信系统的下游任务目标。

```
1 cd semantic_extraction
2 python MLP_MNIST_model.py --epochs 10
3 # 输出模型保存至 saved_model/MLP_MNIST.pkl
```

2.2. 训练语义编码器-解码器

接着训练不同压缩率下的语义编解码器，模拟信道传输过程。

```
1 cd semantic_extraction
2 python MNIST.py
3 # 输出模型保存至 saved_model/MLP_MNIST_coder_[压缩率].pkl
```

2.3. 运行对抗攻击测试

使用 attacks.py 脚本加载训练好的模型，并运行 FGSM、PGD 和 End-to-End 攻击测试。可以通过命令行参数指定压缩率和扰动范围。

```
1 # 回到项目根目录
2 cd ..
3
```

```
4 # 运行完整测试 (包括 FGSM, PGD, E2E)
5 python attacks.py \
6     --compression-rate 0.1 \
7     --epsilons 0.1 0.2 0.3 \
8     --output-dir results/
```

或者直接运行自动化脚本：

```
1 bash run_attacks.sh
```

 Shell

2.4. 生成可视化图表

攻击结束后，结果会保存为 JSON 文件。使用绘图脚本读取数据并生成报告中的图表。

```
1 python generate_charts.py --input results/experiment_results.json
2 # 图表将生成在 results/ 目录下
3 # - attack_comparison.png
4 # - epsilon_curves.png
5 # - psnr_tradeoff.png
```

 Shell